

A Fast Volumetric Capture and Reconstruction Pipeline for Dynamic Point Clouds and Gaussian Splats

Athanasios Charisoudis
athanasios.charisoudis@epfl.ch
École Polytechnique Fédérale de
Lausanne (EPFL)
Lausanne, Switzerland
Lucerne University of Applied
Sciences and Arts
Lucerne, Switzerland

Simone Croci
simone.croci@hslu.ch
Lucerne University of Applied
Sciences and Arts
Lucerne, Switzerland

Lam Kit Yung
kityung.lam@hslu.ch
Lucerne University of Applied
Sciences and Arts
Lucerne, Switzerland

Pascal Frossard
pascal.frossard@epfl.ch
École Polytechnique Fédérale de
Lausanne (EPFL)
Lausanne, Switzerland

Aljosa Smolic
aljosa.smolic@hslu.ch
Lucerne University of Applied
Sciences and Arts
Lucerne, Switzerland

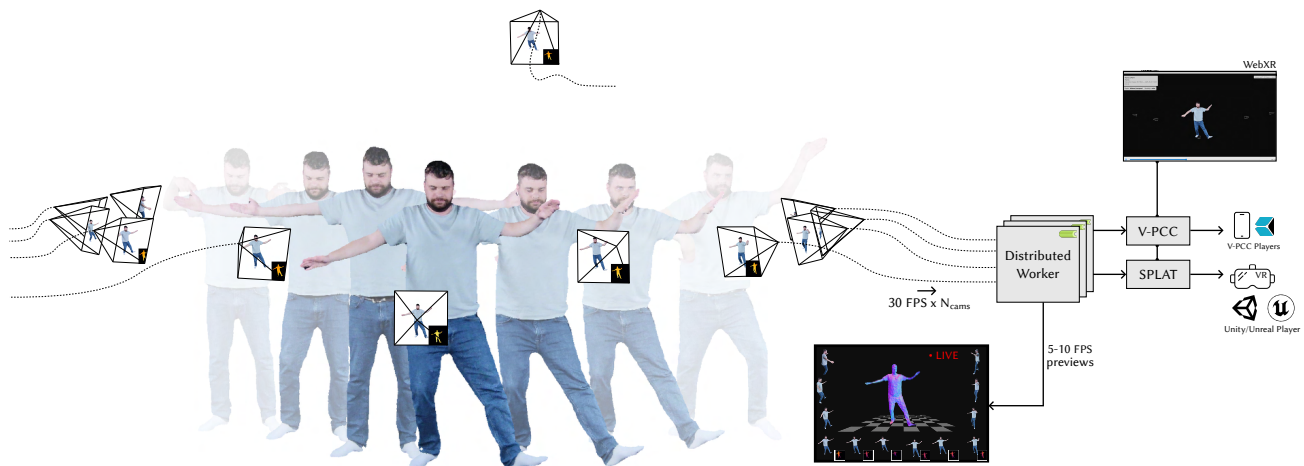


Figure 1: Teaser figure showing the proposed volumetric video capture and reconstruction system on a 12-camera setup.

Abstract

We present a fast and efficient volumetric capture and reconstruction system that processes either RGB-D or RGB-only input to generate 3D representations in the form of point clouds and Gaussian splats. For Gaussian splat reconstructions, we took the GPS-Gaussian regressor and improved it, enabling high-quality reconstructions with minimal overhead. The system is designed for easy setup and deployment, supporting in-the-wild operation under uncontrolled illumination and arbitrary backgrounds, as well as

flexible camera configurations, including sparse setups, arbitrary camera numbers and baselines. Captured data can be exported in standard formats such as PLY, MPEG V-PCC, and SPLAT, and visualized through a web-based viewer or Unity/Unreal plugins. A live on-location preview of both input and reconstruction is available at 5–10 FPS. We present qualitative findings focused on deployability and targeted ablations. The complete framework is open-source, facilitating reproducibility and further research.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
CVMP '25, London, United Kingdom

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-2117-5/2025/12
<https://doi.org/10.1145/3756863.3769713>

CCS Concepts

• **Computing methodologies** → **Point-based models; Image-based rendering; Gaussian Splatting; 3D imaging;** • **Information systems** → *Multimedia content creation.*

Keywords

Volumetric video capture, point clouds, Gaussian splats, dynamic reconstruction

ACM Reference Format:

Athanasios Charisoudis, Simone Croci, Lam Kit Yung, Pascal Frossard, and Aljosa Smolic. 2025. A Fast Volumetric Capture and Reconstruction Pipeline for Dynamic Point Clouds and Gaussian Splats. In *European Conference on Visual Media Production (CVMP '25), December 3–4, 2025, London, United Kingdom*. ACM, New York, NY, USA, 11 pages. <https://doi.org/10.1145/3756863.3769713>

1 Introduction

Volumetric capture promises to carry live performance from stages into immersive media, telepresence, and interactive experiences. Yet going from capture to volumetric video still requires expensive and difficult-to-deploy multi-camera rigs, controlled studio conditions, and heavyweight post-processing [Işık et al. 2023; Pan et al. 2023; Wu et al. 2024a; Zhang et al. 2023; Zheng et al. 2024]. Meanwhile, volumetric systems are increasingly deployed in research and industry, yet streamlined and flexible pipelines that bridge capture, reconstruction, and delivery remain scarce. We therefore present a fast and efficient volumetric capture and reconstruction system that is easy to deploy, supports relatively inexpensive cameras arranged in arbitrary configurations, operates under in-the-wild conditions, and produces volumetric reconstructions suitable for on-location preview and integration with external tools.

In particular, our system handles both RGB-D and RGB-only capture: when depth is present, it performs depth to color alignment and joint bilateral filtering; when depth is absent, it recovers it via calibrated stereo, allowing the system to run with commodity RGB cameras. Moreover, our solution is flexible enough to support any camera configuration, including sparse and irregular camera layouts with variations in camera count, placement, and baseline length; for our experiments we use 6–12 Orbbec Femto Bolt cameras arranged along a semicircular arc facing a small stage, at a mean radius of 2.5 m.

The supported outputs are conventional point clouds suitable for standards-compliant interchange and compression [ISO/IEC JTC 1/SC 29 2025; Turk 1994], and also innovative Gaussian splats, which enable efficient rendering and high-fidelity reconstructions [thr 2025; sup 2025; Kerbl et al. 2023]. The developed pipeline derives both representations from a single, shared preprocessing stream.

The proposed pipeline starts with synchronized multi-view acquisition. Then, we run a unified preprocessing stage comprising performer segmentation, depth to color alignment, optical flow estimation, and image/flow-guided joint bilateral depth filtering on the GPU. From this shared intermediate point, we simultaneously produce (i) per-camera point-cloud reconstructions exported to PLY [Turk 1994] and compressed with MPEG V-PCC [ISO/IEC JTC 1/SC 29 2025], and (ii) per-camera Gaussian splats, exported to PLY and packaged in SPLAT. These output formats are supported by existing players [thr 2025; ant 2025; sup 2025], game engines and various tools. Moreover, we provide a web-based viewer that can display both representations and with WebXR support [Immersive Web Working Group 2025]. In addition, we develop plugins for dynamic Gaussian splat playback in Unity and Unreal Engine 5.

For the generation of Gaussians, we chose to use GPS-Gaussian [Zheng et al. 2024], a feed-forward regressor that converts RGB-D to pixel-wise Gaussians. We assessed that it does not perform well with the tested camera configurations, so we implement some

improvements, namely, the introduction of Gaussian rotation re-parameterization and supervised fine-tuning.

The developed system is modular, throughput-oriented, and standards-compliant. New depth estimators and reconstruction back ends can be added without changing the dataflow.

Our evaluation concentrates on deployability; we present qualitative dynamic reconstruction results and side-by-side visual comparisons across representations and along the processing stages. We also include targeted ablations that isolate the effect of improvements we made to the chosen Gaussian regressor.

The contributions can be summarized as follows.

- We present a fast and efficient volumetric capture and reconstruction system that works with RGB-D or RGB-only input and produces point clouds and Gaussian splats; it is designed for fast setup and in-the-wild conditions, supporting any camera configuration. The system also provides on-location preview at 5–10 FPS.
- We export reconstructions to PLY, MPEG V-PCC, and SPLAT for compatibility with existing standards, and we build a web-based viewer with WebXR support plus plugins for dynamic Gaussian splat playback in Unity and Unreal.
- We improve the GPS-Gaussian regressor by introducing the Gaussian rotation re-parameterization, and fine-tuning on in-domain data, yielding more stable and higher-quality reconstructions across camera configurations.
- We evaluate the proposed volumetric system with dynamic reconstructions, visual comparisons, and ablations.
- We release the codebase of the pipeline, web-viewer and game engines plugins, along with representative samples.

2 Background

Volumetric capture and reconstruction systems exist in both academia and industry, with academic work emphasizing methodological innovation and reproducibility, and commercial solutions focusing on scalable, robust end-to-end pipelines; this section reviews selected examples from both domains.

Among the academic solutions, [Collet et al. 2015] presents a volumetric video pipeline that reconstructs dynamic scenes as textured meshes from multi-camera input and enables efficient compression and streaming for interactive free-viewpoint rendering. LiveScan3D [liv 2025; Kowalski et al. 2015] is an open-source system for real-time 3D reconstruction using multiple Kinect sensors, generating colored point clouds and supporting streaming to Unity and HoloLens for immersive mixed-reality applications. Holoportation [Orts-Escolano et al. 2016] is a real-time 3D capture technology developed by Microsoft Research that reconstructs, compresses, and transmits high-quality 3D models of people, enabling remote users to see, hear, and interact with each other in mixed reality environments. VolumetricCapture [vol 2025c; Sterzentsenko et al. 2018] is a flexible, low-cost, and portable multi-sensor framework that facilitates real-time 3D reconstruction using commodity hardware, supporting applications such as free-viewpoint video and immersive telepresence. The system presented in [Schreer et al. 2019] demonstrates a full volumetric video pipeline, including multi-camera acquisition, geometry and texture reconstruction, and spatio-temporal compression for efficient storage and playback. EasyVolcap [Xu

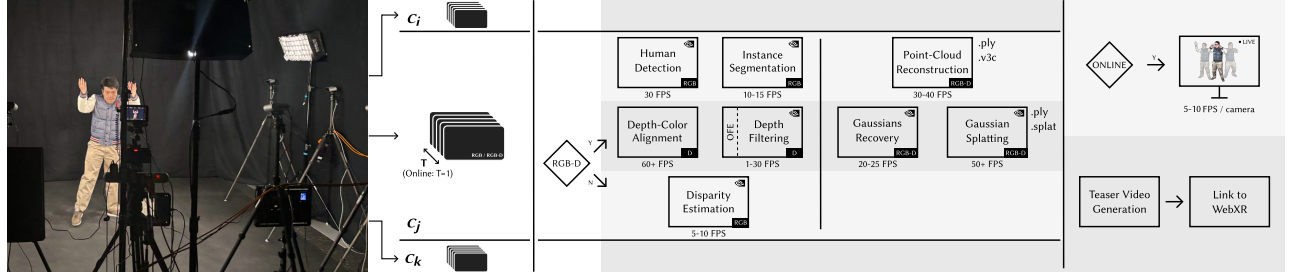


Figure 2: Left: capture in action. Right: pipeline overview (data flow from left to right). Per-camera RGB-D/RGB inputs are processed in parallel; vertical bars indicate synchronization points. Segmentation and depth processing run in parallel, then point-cloud and Gaussian-splat reconstructions are computed in parallel. A live monitor shows reconstruction previews at 5 to 10 FPS, cycling across input cameras; after processing, teaser clips and reconstructions are served in the web viewer.

et al. 2023] is an open-source PyTorch library designed to accelerate neural volumetric video research, providing a unified pipeline for multi-view data processing, 4D scene reconstruction, and dynamic volumetric video rendering. ImViD pipeline [Yang et al. 2025] uses 46 synchronized cameras to capture 5K multi-view video at 60 FPS, enabling high-fidelity volumetric reconstruction for immersive 6-DoF VR experiences. ImViD also integrates spatially aligned audio captured alongside multi-view video, enabling immersive sound rendering.

Of the commercially available solutions, Depthkit Studio [dep 2025] is a portable volumetric video system that enables full-body 360° 3D capture using multiple RGB-D cameras, facilitating real-time streaming and integration into Unity for immersive applications. Evercoast [eve 2025] is a company that provides an end-to-end volumetric video solution enabling the transformation of multi-camera video into high-fidelity 4D spatial data for immersive applications. Their system supports scalable capture setups, ranging from compact to large-scale installations. ScannedReality [sca 2025] offers a streamlined volumetric video solution that utilizes a mobile multi-camera rig, enabling real-time 3D point cloud previews, mesh reconstruction, and seamless deployment across Unity, Unreal Engine, and web platforms. HOLOSYS [4dv 2025] by 4Dviews is a volumetric capture system designed for high-quality 3D video production. It provides seamless integration with Unity, Unreal Engine, and WebXR, facilitating real-time rendering and immersive experiences. Additionally, HOLOSYS+ allows users to choose between mesh-and-texture or Gaussian splat formats. 8i [8i 2025] is a company that offers end-to-end solutions for capturing, transforming, and streaming high-fidelity holograms across various platforms, including VR, AR, and web applications. Their proprietary system utilizes multi-camera capture stages, advanced machine learning algorithms, and real-time compression techniques. Volu [vol 2025a] by Volograms is an AI-powered mobile application that enables users to create, transform, and share dynamic 3D volumetric holograms using just a smartphone, facilitating immersive AR experiences with features like full-body filters and collaborative sharing. Volucap [vol 2025b] is a company specializing in high-resolution volumetric video capture, offering solutions, which utilizes a 42-camera system to produce over 3,000 megapixels per frame, enabling the creation of lifelike 3D human models for applications in virtual reality, augmented reality, and

digital media production. Microsoft’s Mixed Reality Capture Studios [mic 2025] provide a professional volumetric video system that employs dense multi-camera rigs and advanced reconstruction pipelines to generate high-quality, production-ready holographic content.

Compared to the volumetric systems presented before, the proposed volumetric capture and reconstruction system is an academic open-source solution. It can take as input RGB-D or RGB-only and output reconstructions in two possible representations, namely, classical point clouds and innovative Gaussian splats. The system is fast and efficient, offering easy setup and in-the-wild operation, while supporting arbitrary camera configurations, including sparse setups. For playback, we additionally provide a web-based viewer and game engine plugins, together with an on location preview.

3 System Overview

The proposed pipeline receives synchronized multi-view imagery from RGB-D or RGB-only rigs, execute a unified preprocessing stack comprising performer segmentation, depth-color alignment, optical-flow estimation, and image- and flow-guided bilateral filtering on the GPU, and then branch to point-cloud and Gaussian-splat reconstruction backends. The design targets human performance capture with sparse to moderately dense arrays (in the order of 10 cameras) rapid setups in unprepared environments, and therefore operate with low-cost, portable hardware. It should provide low-latency previews suitable for on-site decision making while preserving an offline mode for more expensive processing and improved fidelity. Furthermore, it should scale with the number of sensors and available hardware without reconfiguration, and remain robust to variation in sensor placement and baseline lengths as well as intermittent device or node failures. Outputs support standards-compliant formats, including PLY and MPEG V-PCC for point clouds and PLY and SPLAT for Gaussian splats, while a web-based viewer enables inspection of dynamic sequences. Typical deployments involve compact stages, commodity frame rates and resolutions, and unconstrained illumination and backgrounds. In offline mode, the same preprocessing admits higher-quality variants, including optional color correction and enhanced depth filtering.

3.1 Capture Setup

We employ 6–12 Orbbec Femto Bolt RGB–D sensors arranged along a semicircular arc facing a compact stage area. The camera selection was based on their programmability and depth quality; in qualitative comparisons against two Intel RealSense models and the Azure Kinect DK we observed more stable depth and improved color fidelity. The stage footprint measures approximately 4×4 m, with performers operating near the arc’s center at distances of 0.5–3.5 m, and adjacent cameras were spaced at roughly 0.9 m base-lines. Each device provides time-of-flight (ToF) depth via an infrared emitter/receiver pair, and to mitigate mutual interference we schedule the emitters in a round-robin pattern with a $140 \mu\text{s}$ offset. Camera clocks are synchronized to the host at connection time and re-synchronized every five minutes, enabling timestamp-based alignment in both online and offline processing modes, while hardware trigger and time distribution are supplied via Orbbec SyncHub Pro switches over Ethernet. We terminate a session if inter-camera timestamp drift exceeds 17 ms, ensuring synchronized data recording.

All cameras are mounted rigidly and monitored via onboard inertial measurements to detect micro-motion. In practice, we discard sessions in which the measured angular acceleration exceeds 0.1 rad/s^2 or the linear acceleration exceeds 10^{-3} m/s^2 . Devices are centrally configured from a workstation in a star topology, where a multi-process producer–consumer architecture initializes sensors, enforces manual focus, exposure, and white balance, and orchestrates frame acquisition and storage. Unless noted otherwise, we record color at 3840×2160 and 30 FPS, and depth at 640×576 and 30 FPS; typical settings include an exposure of $120 \mu\text{s}$ and a white balance of 3700 K, with scene-dependent contrast reduction under harsh illumination. Data are buffered in batches, synchronized by timestamps prior to persistence, written in HDF5 containers to local SSD, and mirrored to a network-attached storage array.

Camera calibration techniques are employed to compute the intrinsic and extrinsic parameters for all cameras. For this, we use printed checkerboard and ChArUco patterns that are swept through individual frusta and across multi-view intersections, after which pattern detections in the recorded frames are processed by standard OpenCV-based routines followed by bundle adjustment to recover per-camera parameter and pose matrices. We evaluate two popular open-source libraries, Caliscope [cal 2025] and MultiCamCalib [mul 2025], and adopt the latter owing to its diagnostics and curation tools, which enable detection and removal of high-residual images prior to optimization. When in offline mode, color response is equalized using color-checker targets and three-dimensional LUTs derived in DaVinci Resolve. We use the ColorChecker Video by X-Rite in our experiments. For depth-to-color alignment we rely on factory calibration of the infrared (IR) sensors, which remains stable under controlled temperature conditions used in our indoor captures.

3.2 Pre-Processing Pipeline

The processing pipeline is based on distributed task queues and central orchestration. We use Celery [cel 2025] to define the task graph, and maintain separate queues for the CPU and GPU. At the beginning of each capturing session, we spawn workers for both

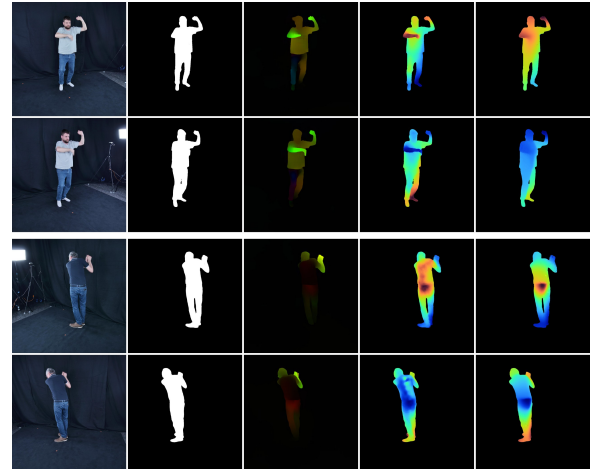


Figure 3: Color cues for two sample image pairs. From left to right: color, mask, optical flow, disparity from RAFT-Stereo [Lipson et al. 2021] and from FoundationStereo [Wen et al. 2025]. RAFT-Stereo was trained on human data only, predicting more accurate disparity ranges.

queues, the number of which depends on the system capability. In our case, we use 16 CPU and 2 GPU workers, as most of our experiments take place in a single-GPU machine. The task nodes and a schematic of the pipeline blueprint is given in Figure 2. To enable robustness, we follow an Extract-Transform-Load (ETL) approach for the node execution, i.e. each node reads its input and stores its output on the disk. Consequently, among the different workers it is only data paths that travel, allowing for task retries in case of failures, at the expense of slightly increased latency.

The pipeline supports online and offline execution modes. In either mode, data flow through mostly the same stages during pre-processing. To increase throughput we split the processing into two stages. During the first stage, each camera stream is processed independently and in parallel with the other streams enabling seamless scaling when more processing nodes or additional compute resources are available. In the second stage, we sync the data among the different cameras, disregarding the ones for which not all cameras were actively recording, and proceed with the dynamic reconstructions. During online mode, the data are already synced, so the syncing step is skipped; otherwise the frame timestamps are used to cluster the data into multi-view frames.

3.2.1 RGB Processing. Initially, the RGB cues are fed to modern human detection and instance segmentation models. After initialization the models remain on the GPU facilitating efficient resource usage; we follow this practice with all deep networks employed in our pipeline. For detecting humans in the initial frames we use YOLO 11 [Jocher and Qiu 2024], followed by SegmentAnythingV2 (SAM2) [Ravi et al. 2024], the state-of-art instance segmentation model at the time of writing. During online mode, we feed the images one-by-one to the segmentor, while in offline mode, a video is created from the frames of each camera. In both cases, the segmentation masks are computed using the video mode of SAM2, which provides temporally consistent human masks and also tracking across frames. This enables us to create human tracks, where in

each track we have the color and corresponding mask for each frame in time. In our setup, SAM2 runs at 10-15 FPS when fed batches of 2 images on an Nvidia RTX 4080 GPU. Given more GPUs, our design allows spawning correspondingly more GPU workers for an approximately analogous speedup in masks generation, thus allowing seamless scaling up.

During offline mode, in parallel to the instance segmentation, we estimate the apparent motion of the pixels for each color sensor independently, which we exploit during subsequent depth filtering tasks. For this, we employ VideoFlow [Shi et al. 2023] the state-of-art multi-frame optical flow estimation (OFE) model at the time of writing of this paper. It uses 3 frames, estimating the optical flow from the two edge ones towards the middle one. Using VideoFlow’s 3-frame prediction network, this node operates at 1-2FPS, making it the bottleneck of our entire processing pipeline and forbidding integration in the online mode. It is important to note here, that we tested other OFE methods, including RAFT [Teed and Deng 2020] and FlowFormer [Huang et al. 2022] which we found inferior. In practice, during the online mode we defer the optical flow computation and depth processing that relies on it for post-capture, to avoid increased latencies unless it is explicitly requested to do else-wise. Figure 3 shows the outputs of the processing steps that each color frame undergoes.

3.2.2 Depth Processing. Given our choice of inexpensive and portable depth sensors, the captured depth cues are usually noisy. This includes normals noise, holes due to sensor errors (phase quantization, IR reflective surfaces etc), and also IR-crosstalk especially when multiple depth sensors are used simultaneously. Consequently, depth cleaning and filtering is vital for having high fidelity reconstructions. In our case depth signal is aligned to color either during or after capturing, where the latter enables more sophisticated interpolation kernels to be used if needed. In both cases, misalignment issues usually remain, especially when the captured depth is at VGA resolution, 6x smaller than the RGB resolution. As a consequence, data denoising is an essential first step to our reconstruction algorithms that rely on sensor depth cues.

To get rid of misaligned depth pixels that are far from the human foreground, we use a quantile-based outlier removal. In particular, we keep all the points inside the foreground that fall in the 99.9th quantile. As a result only misaligned points that belong to the background and when the background is significantly far from the subject (in the order of 1 m or more) are removed. Furthermore, we use a Canny edge erosion on the foreground masks, and invalidate all removed pixels in the depth maps. This helps us minimize the normal noise which is usually concentrated around occlusion boundaries (Figure 5 left).

Finally, we perform guided bilateral filtering [Tomasi and Manduchi 1998] on the aligned yet noisy depth maps. Two such variants were developed. The first uses the color signal as guidance essentially implementing the following equation:

$$D_f(x) = \frac{\sum_{x_i \in \Omega_x} D(x_i) G_{\sigma_s}(\|x_i - x\|) G_{\sigma_r}(\|I(x_i) - I(x)\|)}{\sum_{x_i \in \Omega_x} G_{\sigma_s}(\|x_i - x\|) G_{\sigma_r}(\|I(x_i) - I(x)\|)}$$

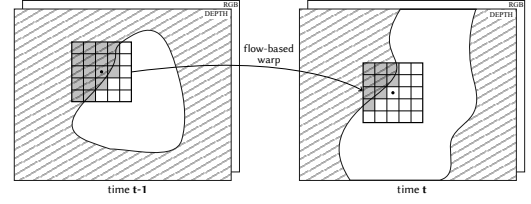


Figure 4: Illustration of how the bilateral spatiotemporal filter is applied to two consecutive frames. The stripes denote valid depth values, while the white regions denote holes.

where D is the noisy depth map and D_f the filtered one, x and x_i denote 2D vectors on the image plane, Ω_x is the neighborhood window centered at x including only valid foreground depth pixels, G_σ is the Gaussian kernel $G_\sigma(x) = \frac{1}{\sigma\sqrt{2\pi}} \exp(-\frac{x^2}{2\sigma^2})$ where σ is the standard deviation, G_{σ_s} and G_{σ_r} are the spatial and range kernels, respectively, and I is the RGB signal. When optical flows are available, we use a newly developed filtering approach, in an attempt to minimize remaining high-frequency depth noise and fill the holes. We first warp the filtered depth D_f^{t-1} and the image I^{t-1} at frame $t-1$ to the current frame t by using the computed (backward) optical flow field obtaining $D_f^{t-1 \rightarrow t}(x)$ and $I^{t-1 \rightarrow t}(x)$, respectively. Then, we filter according to the following equation:

$$D_f^t(x) = \frac{\sum_{x_i \in \Omega_x} D^t(x_i) G_{\sigma_s}(\|x_i - x\|) G_{\sigma_r}(\|I^t(x_i) - I^t(x)\|) + \lambda_t \frac{D_f^{t-1 \rightarrow t}(x) G_{\sigma_t}(\|I^{t-1 \rightarrow t}(x) - I^t(x)\|)}{G_{\sigma_t}(\|I^{t-1 \rightarrow t}(x) - I^t(x)\|)}}{\sum_{x_i \in \Omega_x} G_{\sigma_s}(\|x_i - x\|) G_{\sigma_r}(\|I^t(x_i) - I^t(x)\|) + \lambda_t \frac{D_f^{t-1 \rightarrow t}(x) G_{\sigma_t}(\|I^{t-1 \rightarrow t}(x) - I^t(x)\|)}{G_{\sigma_t}(\|I^{t-1 \rightarrow t}(x) - I^t(x)\|)}}$$

where σ_t is the temporal standard deviation, and λ_t is a weight that controls the importance of the temporal component of the filter.

We call this filter bilateral spatiotemporal (BS+T) as opposed to the bilateral spatial (BS) for the non-temporal version. Using custom CUDA kernels, both filter versions run at over 60 FPS on an RTX 4080. An illustration of how the BS+T is applied to two consecutive RGB-D frames is given in Figure 4. In our experiments, we work with 4K (aligned) depth signals, and use a square window Ω_x of radius $r=7$ pixels (i.e., 15×15), spatial kernel $\sigma_s=7$ pixels, range kernel $\sigma_r=0.1$ (RGB units on a 0–1 scale with ℓ_2 color differences), temporal kernel $\sigma_t=0.06$, and weight $\lambda_t=0.6$ for the BS+T variant; these fixed settings provide stable edge preservation with limited flicker across all the tested sequences. Figure 5 (left block) shows a comparison on real data before and after depth filtering.

3.2.3 Depth from Color. Additionally, the pipeline supports estimating depth from color, which both mitigates sensor–depth failure modes and enables RGB-only operation in environments where active infrared sensing is unreliable. When reliable sensor depth is available this step is skipped.

We compute depth by rectified stereo. For an adjacent camera pair with focal length f (pixels) and baseline B , we rectify the images, infer a disparity field $d(u, v)$, and convert disparity to depth

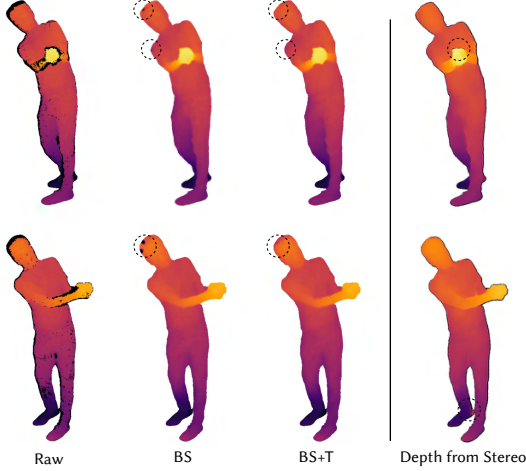


Figure 5: Sensor depth (left block): raw, spatially filtered (BS), and spatio-temporally filtered (BS+T). Stereo-estimated depth (right block) is computed from rectified pairs and shown without bilateral filtering.

via

$$Z(u, v) = \frac{fB}{d(u, v)}.$$

Modern disparity estimation networks are typically trained to predict *positive* disparities for a fixed input ordering (e.g., left–right), which complicates inference when the geometric ordering is reversed. We therefore employ a flipping identity that preserves positivity without retraining. Let $\text{flip}(\cdot)$ denote a horizontal mirror and $d_{L \rightarrow R}$ the disparity predicted on inputs (L, R) . For rectified pairs of width W ,

$$d_{R \rightarrow L}(u, v) = -d_{L \rightarrow R}(W - 1 - u, v) = -\text{flip}(d_{L \rightarrow R})(u, v),$$

so that

$$-\text{flip}(d(L, R)) = d(\text{flip}(L), \text{flip}(R)).$$

This identity restores the positive-disparity convention for either input order, after which $Z = fB/d$ applies unchanged. We apply the flipping patch whenever the median x -coordinate of the foreground in the first image is smaller than that of the second: both inputs are mirrored horizontally, and the resulting disparity is mirrored and negated to enforce the positive-disparity convention.

We evaluate two disparity estimators. *FoundationStereo* [Wen et al. 2025], a state-of-the-art model trained on generic imagery, produces tends to over-smooth human geometry and to merge adjacent limbs with the torso, which hinders reconstruction. *RAFT-Stereo* [Lipson et al. 2021] is retrained following GPS-Gaussian on synthetic THuman2.1 [Yu et al. 2021] renders, except that we generate imagery and ground-truth depth using virtual cameras that match our actual setup rather than the circular layouts used in the default training. This model preserves body part separation and yields satisfactory metric depth in our captures, as can be seen on samples in Figure 5 (right). As with all rectified stereo, applicability degrades as the baseline widens beyond the common field of view; we therefore restrict stereo to adjacent pairs whose overlap remains substantial.

4 Dynamic Reconstruction

From segmented RGB images and aligned depth maps, the pipeline reconstructs a time-ordered sequence of 3D representations without temporal tracking or multi-view fusion, a choice that enables predictable latency and straightforward parallelization in either online or offline modes. Two reconstruction branches run in parallel (Figure 2 middle). The first generates per-camera point clouds that are exported, streamed to the live monitor, and compressed for archival and distribution. The second generates per-camera Gaussian splats whose attributes are predicted from image and depth. The web viewer selects, at each virtual viewpoint, the source camera whose point of view is closest to the virtual camera and switches representations accordingly.

4.1 Point Clouds

Given a calibrated camera i with intrinsic matrix K_i and pose matrix ${}^w_c T_i$ (adopting Craig’s convention [Craig 1987] and writing ${}^b_a T$ for the transformation that maps vectors from coordinate frame a to b), we back-project each foreground pixel (u, v) with depth $d(u, v)$ to a world-space 3D point and attach its color $c_i(u, v)$, thereby producing a colored point cloud per frame and per camera. For a pixel (u, v) with depth d (meters), the camera-space point is

$$c_i \mathbf{x}(u, v, d) = \begin{bmatrix} \frac{(u-c_x)}{f_x} d \\ \frac{(v-c_y)}{f_y} d \\ d \end{bmatrix}, \quad K_i = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix},$$

and the world-space point is ${}^w \mathbf{x}(u, v, d) = {}^w_{c_i} R c_i \mathbf{x}(u, v, d) + {}^w \mathbf{t}_{iORG}$

with ${}^w_c T_i = \begin{bmatrix} {}^w_{c_i} R & {}^w \mathbf{t}_{iORG} \\ \mathbf{0}^T & 1 \end{bmatrix}$ being the pose matrix of camera i .

To suppress residual background leakage and depth speckle, we apply a radius-based validity test in world space. For each point $\mathbf{p}$, we count neighbors within radius r and keep $\mathbf{p}$ only if $|\{\mathbf{q} : \|\mathbf{q} - \mathbf{p}\|_2 < r\}| \geq N_{\min}$. We use $r = 20$ cm and $N_{\min} = 30$, evaluated efficiently with a radius-search K -dimensional tree [Bentley 1975; Friedman et al. 1977], which removes isolated outliers while preserving limb detail at typical working distances.

In RGB-only operation, each adjacent camera pair provides two rectified views and therefore two disparity maps related by the flipping identity described in Subsection 3.2.3. We compute both left–right and right–left disparities, convert each to depth via $Z = fB/d$, and associate the two resulting depth maps back to their originating cameras. Consequently, a physical rig of N cameras yields $2(N - 1)$ depth maps and point clouds per frame. All point clouds are written to disk for archival and compression.

Visualization uses a consistent camera-selection policy for both the live monitor and the web viewer. The virtual camera follows a cubic Bézier path that passes near the physical rig. In sensor-depth mode, the displayed cloud is the one acquired by the physical camera closest to the current virtual viewpoint. In stereo mode, the viewer switches at the virtual position nearest to the current physical camera, transitioning from the previous–current to the current–next stereo cloud so that the displayed geometry remains representative of the local viewpoint.

During online mode, per-camera clouds are serialized to PLY and streamed at interactive rates. We render with a fixed point

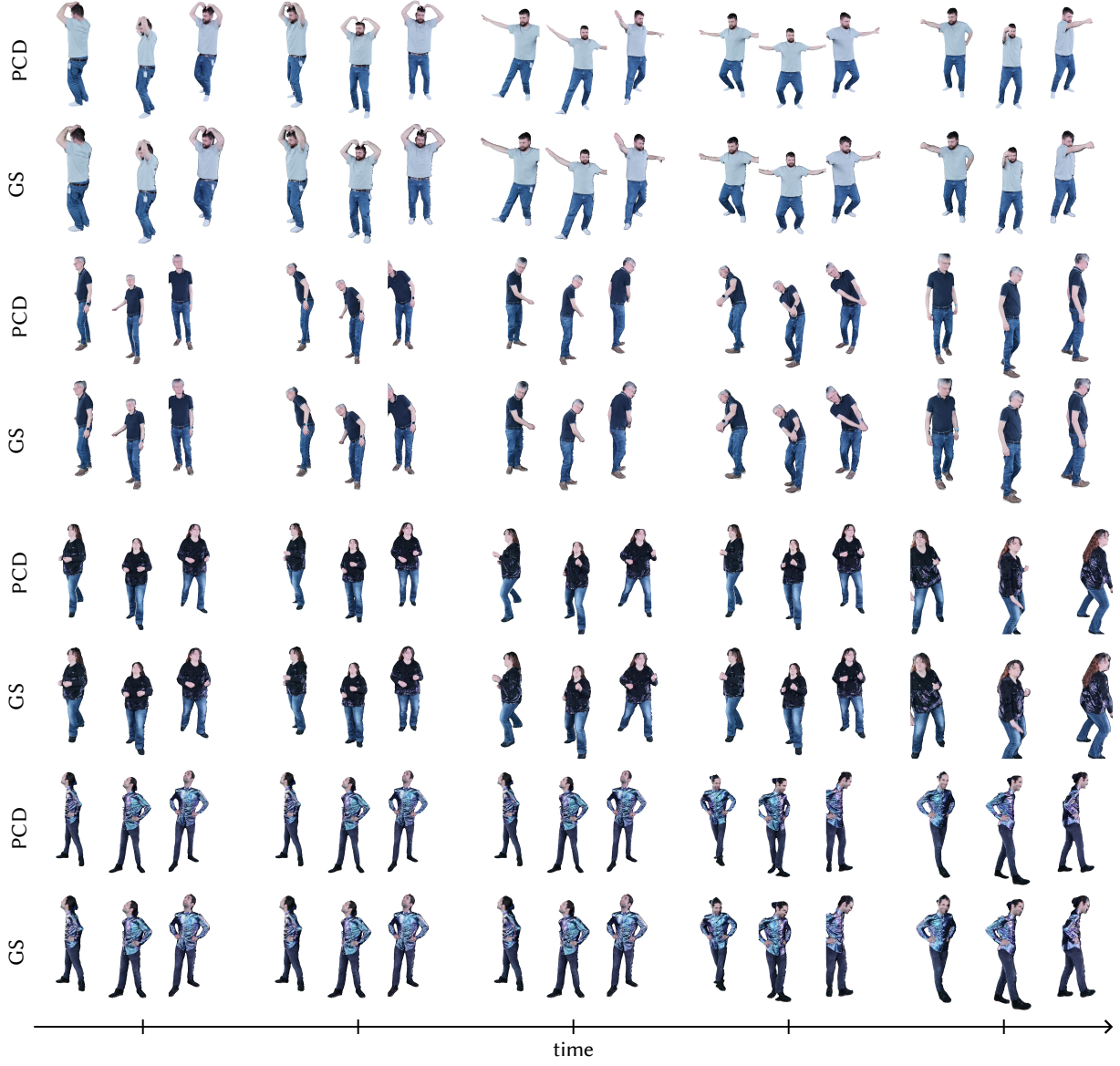


Figure 6: Sample outputs of the dynamic reconstruction pipeline. We randomly select five timesteps for four subjects from the output of the pipeline. For each subject point clouds and Gaussian splat reconstructions are given in triplets, while we visualize three views per triplet.

size to maximize rasterization throughput and to match the projected sampling density at the operating range; at 1024×1024 display resolution and 2.2–2.7 m viewing distances, a constant size of approximately two pixels yields near hole-free coverage without excessive overdraw. A median filter in the depth buffer mitigates quantization artifacts introduced by fixed-size splats. For geometric inspection, we estimate per-point normals by covariance analysis in local neighborhoods: neighbors within a 0.1 m radius (up to 30 points) define a covariance whose minor-axis eigenvector provides the normal direction, yielding stable estimates at commodity sampling densities. In the teaser (Figure 1), we render a sample

reconstruction shaded with per-point normals estimated via the covariance-based method described above.

For archival and distribution, the offline path encodes each per-camera sequence with MPEG V-PCC. We use the reference encoder with common test condition defaults, intra-only groups of pictures for short clips, and default patch packing. In our sequences with approximately 90,000–120,000 points per frame, we observe compression ratios between $3\times$ and $7\times$ depending on scene content; for example, 300 frames occupying roughly 870 MB in PLY compress to about 180 MB. The resulting bitstreams play back in standards-compliant V-PCC players on desktop and mobile devices. In Table 1

Table 1: End-to-end latency and compression for dynamic reconstructions on real captures. Latency is the median per-frame time from sensor input to reconstructed asset emission in the online path (lower is better). Compression is the ratio of encoded size of a camera sequence to raw input (higher is better), using MPEG V-PCC for point clouds and SPLAT for Gaussian splats.

Metric	6 cameras		8 cameras	
	PCD (V-PCC)	GS (SPLAT)	PCD (V-PCC)	GS (SPLAT)
Latency [ms] ↓	130	160	180	230
Compression ratio [×] ↑	5.2	3.2	5.1	3.2

(PCD columns) we report median end-to-end latency and compression for point-cloud reconstructions under the 6- and 8-camera rigs. The online path meets the 5–10 FPS preview target, and the offline encoder achieves $\approx 5\times$ size reduction with the reference V-PCC settings for the selected sequences. Figure 6 (top rows) provides the corresponding qualitative results, showing per-frame point clouds for five subjects at matched times and viewpoints.

4.2 Gaussian Splats

The second reconstruction branch represents each frame as a set of per-camera Gaussian primitives whose positions and colors derive from the corresponding point clouds, while opacity, scale, and rotation are predicted from image evidence and depth using a pixel-wise regression network. This design maintains the per-frame, per-camera granularity that supports low-latency online processing and straightforward parallelization, and it avoids multi-view merging, which we find to degrade in-between viewpoints for sparse rigs.

We parameterize each primitive i by a world-space mean $\mu_i \in \mathbb{R}^3$, a symmetric positive-definite covariance $\Sigma_i \in \mathbb{R}^{3 \times 3}$, a color $c_i \in [0, 1]^3$, and an opacity $\alpha_i \in [0, 1]$. Positions μ_i and colors c_i come from the point cloud back-projection, while the regression network predicts opacity, a rotation $R_i \in \text{SO}(3)$, and axis-aligned scales $s_i = (s_x, s_y, s_z)$ for each valid pixel. The covariance is assembled as $\Sigma_i = R_i \text{diag}(s_i^2) R_i^\top$ in world coordinates. For rendering, a camera with extrinsics (R, t) and intrinsics K projects μ_i to the image plane and induces a screen-space elliptical footprint with covariance $\Lambda_i = J R \Sigma_i R^\top J^\top$, where J is the Jacobian of the perspective projection evaluated at $R(\mu_i - t)$. Contributions at pixel centers are weighted by a Gaussian kernel with covariance Λ_i and composited in approximate front-to-back order using pre-multiplied alphas.

4.2.1 Gaussian Rotations Re-parameterization. During deployment we observed a recurring failure mode in pixel-wise splat regression that arose from a frame inconsistency: several implementations, including the GPS-Gaussian version we evaluated, predict rotations in the *camera* frame and then use them as if they were expressed in *world* coordinates. Circular, same-height camera layouts can hide this error by encouraging highly anisotropic “needle-like” splats that align with the training geometry; arbitrary rigs and non-coplanar camera sets expose the degeneracy as elongated, view-dependent artifacts. We correct this error by

Table 2: Gaussian regressor before/after re-parameterization of Gaussian rotations.

Method	PSNR ↑	SSIM ↑	LPIPS ↓
GS-Regressor from [Zheng et al. 2024]	36.13	0.947	0.049
GS-Regressor world-rot	37.69	0.964	0.048

re-parameterizing the Gaussian rotations in the world frame. To isolate the effect of the rotation change, we train only the Gaussian regressor from [Zheng et al. 2024] and use ground-truth depth as input. Figure 7 shows validation reconstructions before and after the correction.

Let ${}^g\mathcal{R}$ denote the rotation of a Gaussian’s local frame g expressed in the *camera* frame c , as predicted by the regression network, and let ${}^w\mathcal{R}$ be the calibrated camera-to-world rotation. The correct world-frame rotation of the Gaussian (given right-handed rotation systems) is then

$${}^w\mathcal{R} = {}^c\mathcal{R} {}^g\mathcal{R},$$

i.e., the camera-to-world rotation must *premultiply* the predicted camera-frame rotation.

With axis-aligned scales $\mathbf{s} = (s_x, s_y, s_z)$ and $S = \text{diag}(\mathbf{s}^2)$, the world-frame covariance used by the rasterizer becomes

$${}^w\Sigma = {}^g\mathcal{R} S ({}^g\mathcal{R})^\top = {}^c\mathcal{R} ({}^c\mathcal{R} S ({}^c\mathcal{R})^\top) ({}^c\mathcal{R})^\top.$$

Equivalently, if one first forms the camera-frame covariance ${}^c\Sigma = {}^c\mathcal{R} S ({}^c\mathcal{R})^\top$, the change of frame is

$${}^w\Sigma = {}^c\mathcal{R} {}^c\Sigma ({}^c\mathcal{R})^\top.$$

When the network also outputs a camera-frame Gaussian center ${}^c\mu_g$, the corresponding world-space mean is ${}^w\mu_g = {}^c\mathcal{R} {}^c\mu_g + {}^w\mathbf{t}_c$, where ${}^w\mathbf{t}_c$ is the camera origin in world coordinates. In our system the mean position ${}^w\mu_g$ comes from point-cloud back-projection and is already in the world frame; only the rotation, and therefore the covariance, requires re-parameterization.

Using ${}^g\mathcal{R}$ directly as if it were a world-frame rotation, aligns Gaussian principal axes with the camera frame and produces view-point dependent elongations under non-circular layouts. Applying the transformation above removes this pathology. We retrain the Gaussian regressor with the same protocol as GPS-Gaussian, but replace camera-frame rotations by ${}^g\mathcal{R}$ and feed ground-truth depth (skipping RAFT-Stereo module); all other settings remain unchanged. The patched model produces better-conditioned splats with improved sharpness and color fidelity. Quantitative results are reported in Table 2, and qualitative examples in Figure 7, where the retrained regressor is denoted *world-rot*.

4.2.2 Finetuning of GPS-Gaussian. We further align training with deployment by increasing synthetic inter-camera baselines and removing view stitching and multi-view loss aggregation while retaining world-space rotations; these changes reduce geometric bias and yield small improvements. Substantive gains arise from supervised fine-tuning (SFT) performed per camera configuration. We fine-tune on six performances captured with similar configurations, for a total of roughly 72k frames. The objective combines a photometric reconstruction term and a scale regularizer. The

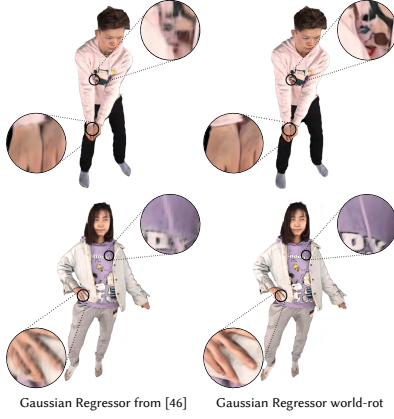


Figure 7: Validation results of the Gaussian regressor with and without Gaussian rotation re-parameterization.

reconstruction loss averages over visible pixels Ω in reprojected images and uses a weighted sum of SSIM [Wang et al. 2004] and the smooth- L_1 (Huber) penalty,

$$\mathcal{L}_{\text{rec}} = \lambda_{L1} \frac{1}{|\Omega|} \sum_{p \in \Omega} \phi_{\delta}(I(p) - \hat{I}(p)) + \lambda_{\text{SSIM}} (1 - \overline{\text{SSIM}}(I, \hat{I})),$$

with $\lambda_{L1} = 0.8$ and $\lambda_{\text{SSIM}} = 0.2$. The Huber penalty (with fixed $\delta = 0.05$) is

$$\phi(e) = \begin{cases} \frac{e^2}{2 \times 0.05}, & |e| < 0.05, \\ |e| - \frac{0.05}{2}, & \text{otherwise.} \end{cases}$$

and $\overline{\text{SSIM}}(I, \hat{I})$ denotes the mean SSIM computed over local 11×11 px windows,

$$\text{SSIM}(x, y) = \frac{(2\mu_x\mu_y + C_1)(2\sigma_{xy} + C_2)}{(\mu_x^2 + \mu_y^2 + C_1)(\sigma_x^2 + \sigma_y^2 + C_2)}.$$

To prevent degenerate anisotropy in the predicted scales, we encourage a high-entropy distribution of scale magnitudes through a differentiable soft-histogram regularizer. Let $\mathbf{s}_i = (s_{x,i}, s_{y,i}, s_{z,i})$ be the per-axis scales of Gaussian i and let $\{b_m\}_{m=1}^M$ be uniformly spaced bin centers on $[0, s_{\max}]$. For axis $a \in \{x, y, z\}$ we form a soft histogram

$$h_{a,m} = \sum_i \exp\left(-\frac{1}{2} \frac{(s_{a,i} - b_m)^2}{\sigma_h^2}\right), \quad p_{a,m} = \frac{h_{a,m}}{\sum_k h_{a,k}},$$

compute the entropy $H_a = -\sum_m p_{a,m} \log(p_{a,m} + \epsilon)$, and minimize a clipped negative entropy,

$$\mathcal{L}_{\text{ent}} = \lambda_{\text{ent}} \sum_a \max(0, H^* - H_a),$$

where H^* is a target entropy threshold, σ_h is the kernel bandwidth, and $\lambda_{\text{ent}} = 10^{-2}$. The full fine-tuning loss is

$$\mathcal{L} = \mathcal{L}_{\text{rec}} + \mathcal{L}_{\text{ent}}.$$

We also modify the scale parameterization to improve transfer across camera distances and intrinsics. Instead of the softplus used

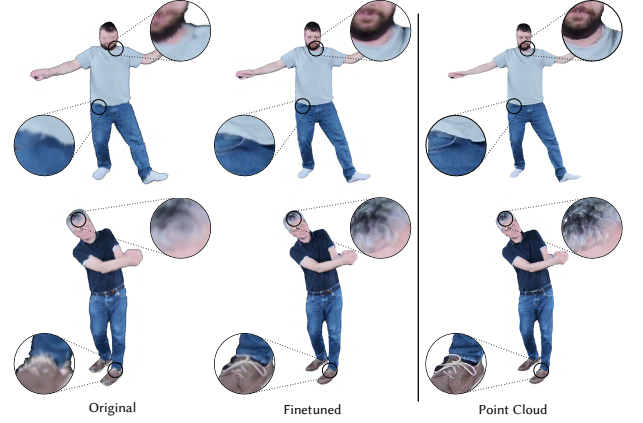


Figure 8: Visual comparisons of Gaussian splat reconstructions before and after fine-tuning, and point clouds.

in GPS-Gaussian, we first bound the pre-activations with a hyperbolic tangent and then apply per-axis instance normalization with learnable affine parameters,

$$\hat{z}_{a,i} = \tanh(z_{a,i}), \quad \bar{z}_{a,i} = \frac{\hat{z}_{a,i} - \mu_a}{\sigma_a}, \quad s_{a,i} = s_{\max}(\gamma_a \bar{z}_{a,i} + \beta_a),$$

where $a \in \{x, y, z\}$, μ_a and σ_a are the per-axis instance statistics, and γ_a, β_a are learned. The covariance uses squared scales $S = \text{diag}(s_{x,i}^2, s_{y,i}^2, s_{z,i}^2)$, which ensures positive definiteness while allowing the affine parameters to re-normalize scale magnitudes during fine-tuning. Empirically, the combination of the SSIM–Huber reconstruction loss and the entropy regularizer, together with the tanh+instance-normalized scale parameterization, produces consistent improvements in robustness and perceived fidelity across venues and motions. Figure 8 presents visual comparisons of Gaussian-splat reconstructions before and after fine-tuning and against the corresponding point-cloud reconstructions; despite these improvements, point clouds remain slightly sharper in most of our experiments.

4.2.3 Rendering and Playback of Gaussian Splats. We encode each frame’s splat data in a standalone SPLAT file and concatenate them into a video SPLAT stream (header with per-frame sizes followed by data blocks), following the spirit of [Fang et al. 2022; Wu et al. 2024b]. During packaging, colors and rotations are quantized to 8 bits and scales to 16–32 bits, which provides the compression reported in Table 1 (SPLAT columns).

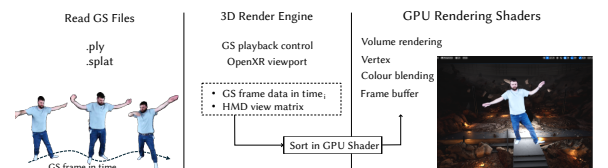


Figure 9: Gaussian Splatting (GS) Video Rendering Pipeline in 3D rendering engines.

For browser playback, we build a three.js/WebGL viewer [thr 2025] with WebXR support that loads PLY, per-frame SPLAT, or

video SPLAT files and plays interactive dynamic reconstructions. Rendering uses one primitive per valid pixel and a screen-space elliptical rasterizer; to reduce aliasing we add a pixel-footprint term ($\approx 0.3 I_2$) to the covariance and enforce a small minimum footprint, similar to [Yu et al. 2024]. We do not prune or build level-of-details (LODs), which keeps primitive counts moderate at our image resolutions. The viewer follows the point-cloud policy for perspective changes, switching to the per-camera Gaussian set closest to the current virtual view.

For Gaussian splat playback in VR and game engines, we additionally provide Unity and Unreal plugins. The general pipeline is given at Figure 9: A 3D object is created in the engine, frame data are streamed and updated over time, and on each frame or camera move the splats are depth-sorted on the GPU. A shader then transforms splats to the current view, projects 3D covariances to 2D, rasterizes and alpha-blends elliptical colors in screen space (frame buffer) to produce the final frame. Unity and Unreal plugins will be made available upon publication.

5 Conclusions

We have presented a volumetric capture and reconstruction system capable of processing RGB-D or RGB-only inputs to produce point clouds and Gaussian splats. By taking the GPS-Gaussian regressor and improving it, our system achieves reliable and efficient Gaussian splat reconstructions. Moreover, it supports in-the-wild operation under uncontrolled illumination and arbitrary backgrounds. Its flexible camera configurations and easy setup and deployment make it suitable for a wide range of capture scenarios. The reconstructions can be exported in standard formats and visualized through web-based and game engine-integrated playback. Moreover, live on-site previews enable immediate feedback. The system has been evaluated and is released open-source, providing a robust platform for further research and applications in volumetric video.

As future work, we plan to further extend and improve the system. We intend to improve both the quality and compression of Gaussian splat reconstructions by adding primitive tracking and bitrate-aware coding of splat attributes.

Acknowledgments

This work was supported by the Swiss Innovation Agency Innosuisse (108.779 INT-ICT, FaVoRe) and the European Commission Horizon programme (GA 101135637, HEAT <https://heat-xr.eu/>).

References

2025. 4Dviews. <https://www.4dviews.com/>
2025. 8i. <https://8i.com/>
2025. Caliscope. <https://github.com/mprib/caliscope>
2025. Celery. <https://docs.celeryq.dev/en/stable/index.html>
2025. DepthKit Studio. <https://www.depthkit.tv/depthkit-studio>
2025. Evercoast. <https://www.evercoast.com/>
2025. GaussianSplats3D. <https://github.com/mkkellogg/GaussianSplats3D>
2025. LiveScan3D. <https://github.com/MarekKowalski/LiveScan3D>
2025. Microsoft mixed reality capture studios. <https://news.microsoft.com/source/features/work-life/microsoft-mixed-reality-capture-studios-create-holograms-to-educate-and-entertain/>
2025. MultiCamCalib. <https://github.com/hjoonpark/MultiCamCalib>
2025. Scanned Reality Studio. <https://scanned-reality.com/>
2025. splat. <https://github.com/antimatter15/splat>
2025. SuperSplat. <https://supersplat.com/>
- 2025a. Volograms. <https://www.volograms.com/>
- 2025b. Volucap. <https://volucap.com/>
- 2025c. Volumetric Capture. <https://vcl3d.github.io/VolumetricCapture/>
- Jon Louis Bentley. 1975. Multidimensional binary search trees used for associative searching. *Commun. ACM* 18, 9 (1975), 509–517.
- Alvaro Collet, Ming Chuang, Pat Sweeney, Don Gillett, Dennis Evseev, David Calabrese, Hugues Hoppe, Adam Kirk, and Steve Sullivan. 2015. High-quality streamable free-viewpoint video. *ACM Transactions on Graphics (ToG)* 34, 4 (2015), 1–13.
- John J. Craig. 1987. Introduction to robotics: Mechanics and control. *IEEE Journal on Robotics and Automation* 3, 2 (1987), 166–166.
- Jiemin Fang, Taoran Yi, Xinggang Wang, Lingxi Xie, Xiaopeng Zhang, Wenyu Liu, Matthias Nießner, and Qi Tian. 2022. Fast Dynamic Radiance Fields with Time-Aware Neural Voxels. In *SIGGRAPH Asia 2022 Conference Papers*.
- Jerome H. Friedman, Jon Louis Bentley, and Raphael Ari Finkel. 1977. An algorithm for finding best matches in logarithmic expected time. *ACM Transactions on Mathematical Software (TOMS)* 3, 3 (1977), 209–226.
- Zhaoyang Huang, Xiaoyu Shi, Chao Zhang, Qiang Wang, Ka Chun Cheung, Hongwei Qin, Jifeng Dai, and Hongsheng Li. 2022. Flowformer: A transformer architecture for optical flow. In *European conference on computer vision*. Springer, 668–685.
- Mustafa İşık, Martin Rünz, Markos Georgopoulos, Taras Khakhulin, Jonathan Starck, Lourdes Agapito, and Matthias Nießner. 2023. HumanRF: High-Fidelity Neural Radiance Fields for Humans in Motion. *ACM Transactions on Graphics (TOG)* 42, 4 (2023), 1–12. doi:10.1145/3592415
- Immersive Web Working Group. 2025. *WebXR Device API*. W3C Candidate Recommendation Draft. World Wide Web Consortium (W3C). <https://www.w3.org/TR/webxr/>
- ISO/IEC JTC 1/SC 29. 2025. Information technology – Coded representation of immersive media – Part 5: Visual volumetric video-based coding (V3C) and video-based point cloud compression (V-PCC). International Standard ISO/IEC 23090-5:2025. <https://www.iso.org/standard/89030.html> Third edition.
- Glenn Jocher and Jing Qiu. 2024. *Ultralytics YOLO11*. <https://github.com/ultralytics/ultralytics>
- Bernhard Kerbl, Georgios Kopanas, Thomas Leimkühler, and George Drettakis. 2023. 3D Gaussian Splatting for Real-Time Radiance Field Rendering. *ACM Transactions on Graphics* 42, 4 (July 2023). <https://repo-sam.inria.fr/fungraph/3d-gaussian-splatting/>
- Marek Kowalski, Jacek Naruniec, and Michal Daniluk. 2015. Livescan3d: A fast and inexpensive 3d data acquisition system for multiple kinect v2 sensors. In *2015 international conference on 3D vision*. IEEE, 318–325.
- Lahav Lipson, Zachary Teed, and Jia Deng. 2021. Raft-stereo: Multilevel recurrent field transforms for stereo matching. In *2021 International Conference on 3D Vision (3DV)*. IEEE, 218–227.
- Sergio Orts-Escolano, Christoph Rhemann, Sean Fanello, Wayne Chang, Adarsh Kowdle, Yury Degtyarev, David Kim, Philip L Davidson, Sameh Khamis, Mingsong Dou, et al. 2016. Holoportation: Virtual 3d teleportation in real-time. In *Proceedings of the 29th annual symposium on user interface software and technology*. 741–754.
- Dongwei Pan, Long Zhuo, Jingtan Piao, Huiwen Luo, Wei Cheng, Yuxin Wang, Siming Fan, Shengqi Liu, Lei Yang, Bo Dai, et al. 2023. Renderme-360: A large digital asset library and benchmarks towards high-fidelity head avatars. *Advances in Neural Information Processing Systems* 36 (2023), 7993–8005.
- Nikhila Ravi, Valentin Gabeur, Yuan-Ting Hu, Ronghang Hu, Chaitanya Ryali, Tengyu Ma, Haitham Khedr, Roman Rädle, Chloe Rolland, Laura Gustafson, Eric Mintun, Junting Pan, Kalyan Vasudev Alwala, Nicolas Carion, Chao-Yuan Wu, Ross Girshick, Piotr Dollár, and Christoph Feichtenhofer. 2024. SAM 2: Segment Anything in Images and Videos. *arXiv preprint arXiv:2408.00714* (2024). <https://arxiv.org/abs/2408.00714>
- Oliver Schreer, Ingo Feldmann, Sylvain Renault, Marcus Zepp, Markus Worchel, Peter Eisert, and Peter Kauff. 2019. Capture and 3D video processing of volumetric video. In *2019 IEEE International conference on image processing (ICIP)*. IEEE, 4310–4314.
- Xiaoyu Shi, Zhaoyang Huang, Weikang Bian, Dasong Li, Manyuan Zhang, Ka Chun Cheung, Simon See, Hongwei Qin, Jifeng Dai, and Hongsheng Li. 2023. VideoFlow: Exploiting temporal cues for multi-frame optical flow estimation. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 12469–12480.
- Vladimiro Stentzenko, Antonis Karakottas, Alexandros Papachristou, Nikolaos Zioulis, Alexandros Doulamanoglou, Dimitrios Zarpalas, and Petros Daras. 2018. A low-cost, flexible and portable volumetric capturing system. In *2018 14th International Conference on Signal-Image Technology & Internet-Based Systems (SITIS)*. IEEE, 200–207.
- Zachary Teed and Jia Deng. 2020. Raft: Recurrent all-pairs field transforms for optical flow. In *European conference on computer vision*. Springer, 402–419.
- Carlo Tomasi and Roberto Manduchi. 1998. Bilateral Filtering for Gray and Color Images. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*. 839–846. doi:10.1109/ICCV.1998.710815
- Greg Turk. 1994. *The PLY Polygon File Format*. Technical Report. Stanford University Computer Graphics Laboratory. <https://gamma.cs.unc.edu/POWERPLANT/papers/ply.pdf> Version 1.0.
- Zhou Wang, Alan C Bovik, Hamid R Sheikh, and Eero P Simoncelli. 2004. Image quality assessment: from error visibility to structural similarity. *IEEE Transactions on Image Processing* 13, 4 (2004), 600–612.
- Bowen Wen, Matthew Trepte, Joseph Aribido, Jan Kautz, Orazio Gallo, and Stan Birchfield. 2025. Foundationstereo: Zero-shot stereo matching. In *Proceedings of the Computer Vision and Pattern Recognition Conference*. 5249–5260.

- Guanjun Wu, Taoran Yi, Jiemin Fang, Lingxi Xie, Xiaopeng Zhang, Wei Wei, Wenyu Liu, Qi Tian, and Xinggang Wang. 2024a. 4d gaussian splatting for real-time dynamic scene rendering. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 20310–20320.
- Guanjun Wu, Taoran Yi, Jiemin Fang, Lingxi Xie, Xiaopeng Zhang, Wei Wei, Wenyu Liu, Qi Tian, and Xinggang Wang. 2024b. 4D Gaussian Splatting for Real-Time Dynamic Scene Rendering. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 20310–20320.
- Zhen Xu, Tao Xie, Sida Peng, Haotong Lin, Qing Shuai, Zhiyuan Yu, Guangzhao He, Jiaming Sun, Hujun Bao, and Xiaowei Zhou. 2023. Easyvolcap: Accelerating neural volumetric video research. In *SIGGRAPH Asia 2023 Technical Communications*. 1–4.
- Zhengxian Yang, Shi Pan, Shengqi Wang, Haoxiang Wang, Li Lin, Guanjun Li, Zhengqi Wen, Borong Lin, Jianhua Tao, and Tao Yu. 2025. ImViD: Immersive Volumetric Videos for Enhanced VR Engagement. In *Proceedings of the Computer Vision and Pattern Recognition Conference*. 16554–16564.
- Tao Yu, Zerong Zheng, Kaiwen Guo, Pengpeng Liu, Qionghai Dai, and Yebin Liu. 2021. Function4d: Real-time human volumetric capture from very sparse consumer rgb-d sensors. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 5746–5756.
- Zehao Yu, Anpei Chen, Binbin Huang, Torsten Sattler, and Andreas Geiger. 2024. Mip-splatting: Alias-free 3d gaussian splatting. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 19447–19456.
- Juze Zhang, Haimin Luo, Hongdi Yang, Xinru Xu, Qianyang Wu, Ye Shi, Jingyi Yu, Lan Xu, and Jingya Wang. 2023. Neurdome: A neural modeling pipeline on multi-view human-object interactions. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 8834–8845.
- Shunyuan Zheng, Boyao Zhou, Ruizhi Shao, Boning Liu, Shengping Zhang, Liqiang Nie, and Yebin Liu. 2024. Gps-gaussian: Generalizable pixel-wise 3d gaussian splatting for real-time human novel view synthesis. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 19680–19690.